10 RAG Architectures

1. Standard RAG

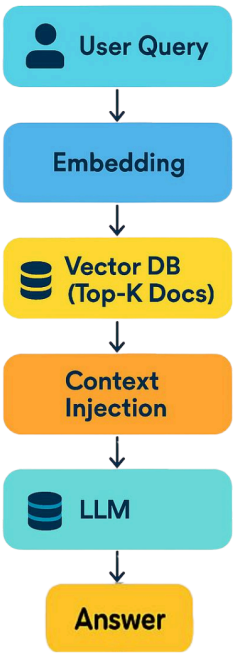
The baseline Retrieval-Augmented Generation setup.

A user query is embedded, relevant documents are retrieved from a vector database, and the LLM generates an answer grounded in that context.

Example

User: *"What are the side effects of ibuprofen?"*
System:

- Retrieves medical documents
- Feeds them to the LLM
- Generates a grounded answer



Usage

- FAQ bots
- Knowledge base assistants
- Documentation search
- Customer support

Best when:

- Knowledge is static or slow-changing
- No multi-step reasoning required

2. Agentic RAG

RAG + autonomous decision-making. The system behaves like an *agent* that can decide:

- what to retrieve
- whether to retrieve
- which tools to call
- how many steps to take

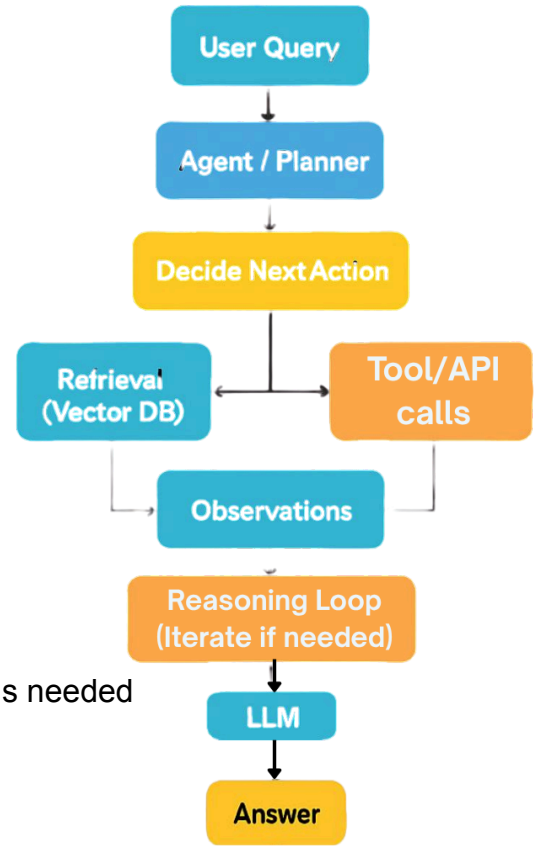
It's iterative and dynamic rather than being a single pass.

Example

User: *"Compare Tesla and BYD revenue growth over the last 3 years."*

Agent:

1. Decides to fetch financial data
2. Calls APIs / retrieves reports
3. May refine query multiple times
4. Aggregates + reasons
5. Produces final comparison



Usage

- Research assistants
- Financial analysis
- Complex Q&A requiring multiple sources
- Autonomous workflows

Best when:

- Multi-step reasoning & Query decomposition is needed
- External tools/APIs are involved

3. RAG with Memory

Extends RAG by incorporating **past interactions or stored user context** into retrieval.
The system remembers:

- conversation history
- user preferences
- past queries

Memory can be:

- short-term (chat history)
- long-term (stored embeddings or profiles)

Example

User:

- Q1: *"I'm a vegetarian."*
- Q2: *"Suggest high-protein foods."*

System:

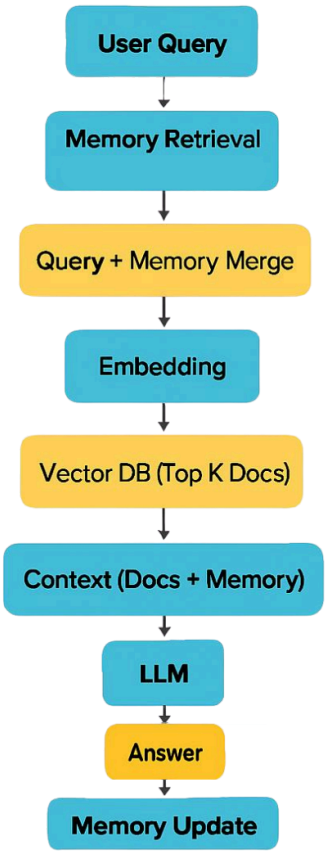
- Retrieves nutrition docs
- Also recalls user preference (vegetarian)
- Filters answer accordingly

Usage

- Personal assistants
- Chatbots with continuity
- Recommendation systems
- Learning companions

Best when:

- Personalization matters
- Conversations span multiple turns
- Context evolves over time



4. Self RAG

The model evaluates its **own output and retrieval quality**.
It can:

- decide whether retrieval is needed
- critique retrieved documents
- refine or re-retrieve before answering

Adds a **self-feedback loop** inside RAG.

Example

User: *"Explain quantum entanglement simply."*
System:

- Retrieves docs
- Generates answer
- Checks: *"Is this clear? sufficient?"*
- If not → retrieves better sources → regenerates

Usage

- High-accuracy QA systems

- Research assistants
- Domains needing **hallucination control**
- Scientific / legal applications

Best when:

- Answer quality matters more than speed
- You want built-in validation

5. Adaptive RAG

The system **adapts retrieval strategy based on query complexity**.
Not every query is treated the same:

- simple => no retrieval or light retrieval
- complex => multi-step retrieval

Example

User:

- *"Capital of France?"* => direct answer (no retrieval)
- *"Impact of inflation on emerging markets?"* => deep retrieval

Usage

- Cost-optimized systems
- Scalable chatbots
- Mixed workloads (simple + complex queries)

Best when:

- You want **efficiency + intelligence**
- Handling large query volumes

6. Corrective RAG

Focuses on **fixing bad retrievals**.
The system:

- detects irrelevant or low-quality documents
- filters or re-ranks them
- may re-query before answering

It improves **retrieval robustness**, not just generation.

Example

User: *"Explain CRISPR gene editing."*
System:

- Retrieves mixed-quality docs
- Detects irrelevant ones
- Filters + re-retrieves
- Produces accurate answer

Usage

- Noisy or unstructured data sources
- Enterprise search systems
- Systems with inconsistent indexing

Best when:

- Retrieval quality is unreliable
- Data sources are messy

7. Attention-based RAG

Uses attention mechanisms to **prioritize the most relevant parts** of retrieved documents.
Instead of treating all retrieved chunks equally, the model:

- assigns weights to different passages
- focuses on high-signal content during generation

Example

User: *"Causes of climate change?"*
System:

- Retrieves multiple documents
- Uses attention to emphasize key sections (e.g., greenhouse gases)
- Generates answer based on most relevant snippets

Usage

- Long-document QA
- Summarization systems
- Legal / research analysis

Best when:

- Retrieved context is large or noisy
- Fine-grained relevance matters

8. HybridAI RAG

Combines **neural methods (embeddings/LLMs)** with **symbolic or rule-based systems** (knowledge graphs, logic rules).
It enables:

- structured reasoning
- better interpretability
- improved factual consistency

Example

User: *"Who is the CEO of the company that owns Instagram?"*
System:

- Uses knowledge graph to resolve relationships
- Uses LLM for explanation
- Produces accurate, structured answer

Usage

- Enterprise knowledge systems
- Compliance / regulated domains
- Complex relational queries

Best when:

- Structured + unstructured data both matter
- Logical reasoning is required

9. Cost-Constrained RAG

Optimizes RAG pipelines under **cost constraints** (tokens, API calls, latency).
The system dynamically:

- limits retrieval depth
- chooses cheaper models when possible
- balances cost vs quality

Example

User: *"Summarize this document."*
System:

- Checks budget
- Uses fewer retrieved chunks or smaller model
- Produces acceptable answer within cost limits

Usage

- Production systems at scale
- SaaS AI products
- High-volume query environments

Best when:

- Cost efficiency is critical
- Trade-offs between quality and expense are needed

10. XAI RAG (Explainable AI RAG)

Focuses on **making RAG outputs explainable**.
The system:

- shows retrieved sources
- provides reasoning traces
- justifies answers with evidence

Example

User: *"Why was this loan rejected?"*
System:

- Retrieves policy documents
- Generates answer

- Explains decision with cited rules and reasoning

Usage

- Finance / healthcare
- Legal systems
- Auditable AI applications

Best when:

- Transparency is mandatory
- Decisions must be justified